LABORATORIO 2

ALU: lógica combinacional y diseño jerárquico

Parte A: diseño y verificación de una ALU en VHDL · Parte B: reestructuración jerárquica con componentes

---

Desarrollar en grupos de máximo 3 personas

Profesor: Diego Quiroga · Docente de inicio: Yazmín Bentancour

Entrega: ___ /___ · Reporte de realización del laboratorio (formato IEEE)

Nombre y Apellido: _____	C.I.: _____
Nombre y Apellido: _____	C.I.: _____
Nombre y Apellido: _____	C.I.: _____
Nombre del equipo: _____	Fecha: ___ / ___ / ____

Propósito

Este laboratorio tiene como propósito diseñar, describir en VHDL, verificar e implementar sobre la placa una Unidad Aritmético-Lógica (ALU) combinacional de 16 operaciones, abordándola en dos partes. En la Parte A la ALU se describe como un único bloque combinacional; el foco está en el diseño de un circuito combinacional complejo a partir de una especificación, el uso de tipos numéricos para la aritmética y las banderas de estado. En la Parte B se retoma ese mismo diseño para reestructurarlo con enfoque jerárquico, en un conjunto de módulos más pequeños que se conectan mediante instanciación de componentes, con el objetivo de comprender las ventajas de la modularidad y la reutilización y aprender a verificar un sistema por partes. El comportamiento de la ALU es el mismo en ambas partes; lo que cambia es cómo se organiza el diseño.

Materiales

- Spartan Edge Accelerator Board (SEA) — FPGA Xilinx Spartan-7 XC7S15.
- Shield multifunción UTEC (12 dip-switches como entradas y 10 LEDs como salidas).
- Cable USB-C y programador JTAG (HS2).
- Manual de pines de la SEA y del shield UTEC (Moodle).

Software

- Xilinx Vivado (versión 2025.1). Opcional: EDA Playground.

Objetivos del laboratorio

Parte A — ALU combinacional

- Diseñar una ALU a partir de una especificación dada por su tabla de operaciones.
- Describir en VHDL operaciones aritméticas, lógicas y de desplazamiento utilizando el paquete numeric_std.
- Implementar y utilizar banderas de estado (Carry y Zero).
- Verificar exhaustivamente el diseño mediante un testbench que cubra todas las operaciones.
- Implementar y validar la ALU en hardware, con entradas por dip-switches y salidas por LEDs.

Parte B — Diseño jerárquico

- Aplicar el diseño jerárquico: descomponer un sistema en módulos y conectarlos mediante componentes (component / port map).
- Reutilizar un mismo módulo (un sumador) en varias operaciones, evidenciando el valor de la modularidad.

- Organizar el diseño con una metodología top-down / bottom-up.
- Verificar el diseño por partes: un testbench por módulo y un testbench del sistema completo.
- Implementar y validar la ALU jerárquica en hardware.

1. La placa: entradas y salidas

La ALU recibe todas sus entradas por un único bus SW de 12 bits, que internamente se separa en A, B y opcode. Las salidas Result, Carry y Zero se muestran en 6 LEDs. Recuerde que los dip-switches se alimentan del 3.3 V del header Arduino, que debe habilitarse manejando el pin AR_3V3_EN en '0' (visto en el Laboratorio 1).

Organización del bus de entrada

Bits del bus	Señal
`SW(3 downto 0)`	Operando A (4 bits)
`SW(7 downto 4)`	Operando B (4 bits)
`SW(11 downto 8)`	Opcode (4 bits)

Asignación de pines

Señal	Bus / LED
A(0..3)	SW(3..0)
B(0..3)	SW(7..4)
opcode(0..3)	SW(11..8)
Result(0..3)	4 LEDs
Carry	1 LED
Zero	1 LED
AR_3V3_EN	(habilita 3.3 V)

Recuerde: Todo diseño que use los dip-switches debe incluir la salida AR_3V3_EN y forzarla a '0' (pin L13); de lo contrario no hay 3.3 V y las entradas se leen siempre como '0'. Estándar de E/S: LVCMOS33.

2. Parte A — ALU combinacional (descripción monolítica)

En esta parte la ALU se describe como un único bloque combinacional. En la Parte B se reutilizará este mismo diseño para reestructurarlo con enfoque jerárquico.

A.1 — Especificación de la ALU

Implemente una ALU con operandos A y B de 4 bits (sin signo) y las 16 operaciones del Cuadro 1, seleccionadas por el opcode de 4 bits. La salida es Result (4 bits) más las banderas Carry y Zero.

Opcode	Operación	Descripción
0000	**A + B**	Suma
0001	**A − B**	Resta (el Carry indica borrow)
0010	**A AND B**	AND lógico bit a bit
0011	**A OR B**	OR lógico bit a bit
0100	**A XOR B**	XOR lógico bit a bit
0101	**NOT A**	Negación lógica de A
0110	**A << 1**	Desplazamiento lógico a la izquierda
0111	**A >> 1**	Desplazamiento lógico a la derecha

Opcode	Operación	Descripción
1000	**A + 1**	Incremento de A
1001	**A − 1**	Decremento de A
1010	**B + 1**	Incremento de B
1011	**B − 1**	Decremento de B
1100	**comp2(A)**	Complemento a 2 de A
1101	**comp2(B)**	Complemento a 2 de B
1110	**A = B**	Comparación: 1 si A = B, 0 en caso contrario
1111	**A > B**	Comparación: 1 si A > B, 0 en caso contrario

A.2 — Diseño en VHDL

Escriba la ALU en un archivo alu.vhd. La entidad ya está definida por la interfaz siguiente (respétela para que coincida con la asignación de pines):

```
entity alu is
    Port ( SW        : in  STD_LOGIC_VECTOR(11 downto 0);
           Result    : out STD_LOGIC_VECTOR(3 downto 0);
           Carry     : out STD_LOGIC;
           Zero      : out STD_LOGIC;
           AR_3V3_EN : out STD_LOGIC );
end alu;
```

En la arquitectura, separe el bus en señales A, B y opcode, y describa las operaciones con un proceso y una sentencia case. A modo de guía se muestra el esqueleto con una operación resuelta; complete el resto:

```
A      <= unsigned(SW(3 downto 0));
B      <= unsigned(SW(7 downto 4));
opcode <= SW(11 downto 8);

process(A, B, opcode)
    variable r : unsigned(3 downto 0);
begin
    r := (others => '0');
    case opcode is
        when "0010" => r := A and B;   -- ejemplo (AND)
        -- ... completar las 15 operaciones restantes ...
        when others => r := (others => '0');
    end case;
    -- asignar Result, Carry y Zero
end process;
```

Sobre las banderas: La suma (0000) es Result = A + B, y el Carry es el acarreo de salida. En la resta (0001) el Carry representa el borrow. La bandera Zero se activa cuando Result es 0000. Para las operaciones lógicas y de comparación, el Carry se define en 0.

- Use el paquete numeric_std (tipo unsigned) para la aritmética.
- Incluya la asignación AR_3V3_EN <= '0'.
- Asegúrese de asignar Result, Carry y Zero en todas las ramas (o con valores por defecto) para no inferir latches.

A.3 — Simulación

- Escriba un testbench que ejercite las 16 operaciones, con varios valores de A y B en cada una (incluya casos de borde: acarreo, resultado cero, A < B, A = B).
- Se recomienda un testbench autoverificable: calcular el resultado esperado de forma independiente y compararlo con la salida del diseño mediante assert.
- Ejecute la simulación funcional en Vivado o EDAplayground y capture las formas de onda para el informe.

A.4 — Implementación en la placa

- Cree el archivo de restricciones (.xdc) según la asignación de pines de la sección 1 (IOSTANDARD LVCMOS33), incluyendo AR_3V3_EN en el pin L13.
- Genere el bitstream, programe la placa por JTAG y valide el funcionamiento accionando los dip-switches (A, B y opcode) y observando los LEDs (Result, Carry, Zero).
- Grabe un video mostrando al menos 10 operaciones distintas, con al menos 3 valores de entrada en cada una.

3. Parte B — Diseño jerárquico de la ALU

La entidad de la ALU, el bus de entrada y la asignación de pines son los mismos de la Parte A (sección 1): la interfaz no cambia, solo cambia la organización interna del diseño.

B.1 — Descomposición en módulos

Reestructure la ALU de la Parte A (las mismas 16 operaciones y la misma tabla de opcodes) con un enfoque jerárquico. Se sugiere la siguiente descomposición en módulos, cada uno en su propio archivo VHDL con entidad y arquitectura:

Módulo	Rol	Entradas	Salidas
`adder4`	Sumador de 4 bits reutilizable	a, b, cin	s, cout
`arith_unit`	Suma, resta, ±1, comp2 (usa adder4)	A, B, opcode	result, carry
`logic_unit`	AND, OR, XOR, NOT	A, B, sel	result
`shift_unit`	Desplazamientos de 1 bit	A, dir	result, carry
`compare_unit`	Comparación A=B y A>B	A, B	eq, gt
`alu (top)`	Conecta todo: mux, cero y carry	SW(11..0)	Result, Carry, Zero, AR_3V3_EN

La jerarquía resultante tiene la siguiente forma (note los dos niveles: la ALU contiene la unidad aritmética, y esta a su vez contiene el sumador):

```
alu  (top, arquitectura estructural)
  |-- arith_unit  ----->  adder4   (sumador REUTILIZADO)
  |-- logic_unit
  |-- shift_unit
  |-- compare_unit
  |-- multiplexor de seleccion (segun opcode)
  |-- deteccion de cero
  '-- seleccion de carry
```

B.2 — Diseño de los módulos

Implemente cada módulo por separado. El bloque central es un sumador reutilizable; a modo de ejemplo se muestra su interfaz:

```
entity adder4 is
    Port ( a, b : in  unsigned(3 downto 0);
           cin  : in  std_logic;
           s    : out unsigned(3 downto 0);
           cout : out std_logic );
end adder4;
```

Idea clave (reutilización): No hace falta un módulo distinto para restar, incrementar o hacer complemento a 2: todas esas operaciones se obtienen reutilizando el sumador con los operandos adecuados. Por ejemplo, A − B = A + (NOT B) + 1, y el complemento a 2 de A es (NOT A) + 1. La unidad aritmética selecciona los operandos según el opcode y usa una única instancia del sumador.

B.3 — Integración: el top estructural

Escriba el archivo alu.vhd (mismo nombre de entidad e interfaz que en la Parte A). Su arquitectura debe ser estructural: declarar los componentes, instanciarlos con port map, y agregar el multiplexor que selecciona el resultado según el opcode, la detección de cero y la selección del carry.

```
architecture Structural of alu is
    component arith_unit ... end component;
    component logic_unit ... end component;
    -- ... (demas componentes)
    signal ar_res, lo_res, sh_res : unsigned(3 downto 0);
    -- ... (senales internas)
begin
    U_ARI : arith_unit port map ( ... );
    U_LOG : logic_unit port map ( ... );
    -- ... (demas instancias)
    -- multiplexor de seleccion, deteccion de cero y seleccion de carry
end Structural;
```

B.4 — Verificación modular

- Escriba al menos un testbench por módulo para verificar cada bloque de forma aislada (por ejemplo, un testbench exhaustivo del sumador).
- Verifique el sistema completo: puede reutilizar el testbench de la ALU de la Parte A, ya que la interfaz no cambió. Debe dar los mismos resultados.
- Simule en Vivado y capture las formas de onda para el informe.

B.5 — Implementación en la placa

- Genere el bitstream, programe la placa por JTAG y valide el funcionamiento accionando los dip-switches y observando los LEDs.
- Grabe un video mostrando al menos 10 operaciones distintas, con al menos 3 valores de entrada en cada una.

4. Preguntas para el informe

Parte A — ALU combinacional

- La ALU es un circuito puramente combinacional. ¿Qué caracteriza a una descripción combinacional en VHDL y qué precauciones hay que tomar para no inferir latches accidentalmente (por ejemplo, en un proceso con case)?
- Explique el significado de las banderas Carry y Zero. En particular, en A − B, ¿qué representa el Carry? ¿Y en A + B cuando el resultado excede 4 bits?
- ¿Por qué conviene usar el paquete numeric_std (tipos unsigned/signed) para la aritmética en lugar de operar directamente sobre std_logic_vector?
- Verificación: ¿cómo puede garantizar que las 16 operaciones funcionan correctamente? Comente la estrategia de su testbench (cobertura de casos, uso de un modelo de referencia, casos de borde).
- Proponga una modificación para extender la ALU (por ejemplo, a operandos de 8 bits, o agregar una operación nueva). ¿Qué partes del diseño deberían cambiar y cuáles no?

Parte B — Diseño jerárquico

- Comente las ventajas y desventajas del diseño jerárquico frente a una descripción monolítica como la de la Parte A.
- La unidad aritmética reutiliza un único sumador para varias operaciones. Explique cómo se implementan la resta, los incrementos/decrementos y el complemento a 2 reutilizando el sumador, y qué ventaja tiene esa reutilización.
- Verificación: ¿qué ventaja aporta verificar cada módulo por separado además del sistema completo? ¿Cómo ayuda esto a localizar errores?
- ¿Qué diferencia hay entre las metodologías top-down y bottom-up de diseño? ¿Cuál siguió en este laboratorio y por qué?

- Proponga una estrategia para convertir esta ALU en un microprocesador simple: ¿qué bloques adicionales necesitaría y qué instrucciones de ensamblador podría implementar?

5. Entregables

El desarrollo debe ser original, por grupos de un máximo de 3 personas. Los trabajos idénticos reportados por separado serán causal de anulación de todos los grupos involucrados. Deberá entregar:

- Código fuente VHDL completo de la Parte A: ALU monolítica y su testbench.
- Código fuente VHDL completo de la Parte B: todos los módulos, el top estructural y los testbenches.
- Resultados de la simulación: de la ALU completa en la Parte A, y por módulo y del sistema completo en la Parte B.
- Un video de la simulación y funcionamiento en la placa por cada parte: al menos 10 operaciones, con al menos 3 valores de entrada cada una.
- Un único informe en formato IEEE que abarque ambas partes y responda las preguntas de la sección 4, incluyendo el diagrama de la jerarquía del diseño de la Parte B. (Plantilla formato IEEE LaTeX)

6. Rúbrica de evaluación

La evaluación utiliza la rúbrica por niveles de logro de la asignatura (Destacado / Competente / En desarrollo / Inicial); consulte el documento de rúbrica para los descriptores de cada nivel. Los criterios y sus pesos son: Cada criterio se evalúa considerando ambas partes del laboratorio.

Criterio	Peso
Funcionamiento de la ALU en la placa	30 %
Simulación (testbench)	20 %
Video de testing (simulación e implementación física)	10 %
Informe: claridad y organización	15 %
Informe: preguntas específicas	15 %
Informe: conclusiones	10 %

ES OBLIGATORIO PRESENTAR EL FUNCIONAMIENTO DEL CIRCUITO DE FORMA PRESENCIAL, coordinando previamente con los docentes.

En la rúbrica de evaluación por niveles de logro encontrarán el detalle de lo que se espera para cada uno de los puntos de la tabla.